BREVE RESUMEN “TIPO MANUAL” DE PROGRAMACIÓN ORIENTADA A OBJETOS

Resumen introductorio del libro “Programación en Java 6” **PARTE I**
Luis Joyanes Aguilar, Ignacio Zahonero Martínez

Implementaciones de algunos ejemplos en código Python **PARTE II**

Resumen desarrollado por: Alexander Nina *v1*

CONTENIDO

PARTE I 2
1. Paradigma estructurado 2
1.1. Razones. 2
2. Programación orientada a objetos POO 2
2.1. Conceptos fundamentales de la programación orientada a objetos 2
2.1.1. Abstracción 2
2.1.2. Encapsulamiento y ocultación de datos 2
2.1.3. Herencia 3
2.1.4. Polimorfismo 3
2.1.5. Reutilización (reusabilidad) 3
2.2. Clases 3
2.3. Objetos 4
2.3.1. Estado 4
2.3.2. Comportamiento 4
2.3.3. Identidad 4
2.3.4. Cómo identificar las clases y objetos 4
2.3.5. Relaciones entre clases 4
2.3.6. UML: Modelado de aplicaciones 4
a) Modelo: 4
¿Qué es UML? 5
2.3.7. Diseño y representación gráfica de clases y objetos en UML 5
2.3.8. Representación gráfica de una clase 5
PARTE II 6
1. Declaración de una clase 6
1.1. Constructor 6
1.2. Self 6
2. Ejemplo 1 Clase Punto 6
2.1. Dar nombre a la clase. 6
2.2. Constructor y Atributos. 6
2.3. Método mostrar coordenada X e Y. 7
2.4. Modificación de coordenadas una vez instanciado el objeto. 7
3. Visibilidad de los miembros de la clase 7
3.1. ¿Cuándo usar privado, público y protegido? 7
3.2. ¿Cómo obtengo u modifico el valor de un atributo? 8
4. Métodos de una clase 8
4.1. Ejemplo 2 Fecha de cumpleaños 8
5. Paquetes y Clases predefinidos en Python 9
6. Tipos de datos Abstractos 9
7. Herencia 10

PARTE I

1. Paradigma estructurado

Hasta ahora vimos una metodología de programación o paradigma denominado estructurado, el cual data de 60 y 70's desde la creación del Pascal por Niklaus Wirth, este enfoque genera programas bien escritos y legibles; con esta metodología se siguen reglas y técnicas. Las **reglas** de diseño estructurado se basan en la modularización y que cada módulo obtenga una solución individual lo que implica un diseño descendente, las **técnicas** incluyen estructuras o instrucciones básicas de control: *Secuencia, Decisión o Selección y Bucles o Iteraciones*. Las estructuras indican el orden en el que se ejecutan las instrucciones de un algoritmo, una vez resueltos los subproblemas o módulos se combinan para resolver el problema global.

En problemas pequeños esto funciona y resulta eficiente, creamos la lista de instrucciones en un lenguaje y las ejecutamos; **pero** cuando los problemas se vuelven más grandes a medida que aumenta la complejidad del problema, así aumentan las instrucciones y se vuelve difícil de controlar.

Es normal el poder controlar algunos centenares de líneas de instrucciones; para solucionar el problema los programas se desglosan en unidades más pequeñas que según el lenguaje, adoptan el nombre de funciones, métodos, procedimientos, subprogramas o subrutinas, así se puede dividir en *funciones*, cada función resolviendo una tarea en específico, con el tiempo esta idea evolucionó y su agrupamiento llegó a unidades más grandes llamadas módulos, el principio el mismo "*agrupar componentes que ejecutan listas de instrucciones o sentencias*", así conforme los programas se hacen más grandes y complejos, el paradigma estructurado comienza a dar señales de debilidad y resulta difícil terminar los programas de un modo eficiente.

1.1.Razones: el acceso ilimitado a los datos globales por parte de las funciones y que los fundamentos del paradigma procedimental, como funciones inconexas y datos proporcionan un modelo pobre del mundo real.

2. Programación orientada a objetos POO

Este tipo de programación se compone de objetos, elementos autosuficientes de un programa que representa un grupo de características relacionadas entre sí y diseñado para realizar una tarea dada; cada objeto con una funcionalidad específica expuesta a sus usuarios y una implementación *oculta*.

Para problemas pequeños, la división en módulos puede funcionar bien; en problemas más grandes o complejos, los objetos funcionan mejor.

DOO o diseño orientado a objetos el primer paso en el proceso de resolución de problemas es identificar los componentes u objetos que forman el soporte de la solución y determinar cómo interactúan entre sí.

Los objetos constan de datos y operaciones que se realizan sobre esa información, cada objeto combina en una única entidad o componente, datos y operaciones llamados funciones o procedimientos y métodos.

El paso siguiente requiere encapsular los datos y operaciones que manipulan esos datos en una entidad única en los lenguajes de programación es una plantilla o modelo que permite crear objetos "clase".

2.1. Conceptos fundamentales de la programación orientada a objetos

La programación orientada a objetos (POO) es un enfoque conceptual específico para diseñar programas usando un lenguaje que se centra en los objetos como C++, Java o Python cuyas propiedades más importantes son:

- Abstracción
- Encapsulamiento y ocultación de datos
- Herencia
- Polimorfismo
- Reusabilidad o reutilización de código

Siendo los 4 primeros los famosos "pilares de la programación orientada a objetos", las limitaciones encontradas en el paradigma estructurado se superan en este paradigma. POO tiene los elementos más fundamentales como clases y objetos.

Lo que buscaremos siempre en este paradigma es un objeto tal como lo percibe una persona, pensando en los datos necesarios para describirlo y las operaciones que permitirán la iteración del usuario con los datos para después desarrollar una descripción de la interfaz externa y decidir cómo implementarla y almacenar los datos, finalmente juntarlos en un programa que lo use cualquier persona.

2.1.1. Abstracción

La propiedad que considera los aspectos más significativos de un problema y expresa una solución en esos términos, es la etapa crucial de **representación de la información** en relación con la interfaz y el usuario; luego se representa con el diseño de una **clase**.

a) Clase. - Una clase es un elemento que traduce una abstracción a un tipo definido por el usuario y combina representaciones de *datos* y *métodos* para manipular esa información.

La abstracción posee diversos grados o niveles que ayudan a estructurar la complejidad que poseen los sistemas del mundo real.

Ejemplo: la televisión es un electrodoméstico que se encuentra en todos los hogares, la mayoría puede conocer sus características y su manejo manual o con un control remoto: encender, apagar, cambiar de canal, ajustar el volumen, cambiar el brillo y mucho más, también en una televisión podemos añadir componentes externos: reproductores de DVD, *home theaters*, decodificadores de señal (televisión por cable). PERO ¿Cómo funciona internamente?, ¿Cómo se recibe la señal de cable o satélite y se puede reproducir en la pantalla? No sabemos cómo funciona una televisión, pero si sabemos usarlo; la televisión separa su implementación interna con su interfaz externa (*lo que podemos usar*).

Nota. - La abstracción es el principio fundamental que se encuentra tras la reutilización. Sólo se puede reutilizar un componente o elemento si en él se ha abstraído la escena de un conjunto de elementos del mundo real en el que aparecen una y otra vez con ligeras variantes en sistemas diferentes.

2.1.2. Encapsulamiento y ocultación de datos

Significa reunir en cierta estructura todos los elementos que, a un nivel de abstracción, se pueden considerar de una misma entidad y es el proceso de agrupamiento de datos y operaciones relacionadas bajo una misma unidad de programación. En este caso, los objetos que poseen las mismas características y comportamiento se agrupan en clases que son unidades de programación que encapsulan datos y operaciones; la encapsulación *oculta* lo que hace un objeto de lo que hacen otros objetos del mundo exterior; por lo que se denomina *ocultación de datos*.

Ejemplo: Un objeto tiene que presentar una "cara" al mundo exterior (interfaz), de ese modo se puede iniciar operaciones; como la televisión tiene botones que al presionarlos activan ciertas acciones, una lavadora tiene un conjunto de mandos que establecen la temperatura y el nivel del agua; los botones de la TV y los mandos de la lavadora constituyen la comunicación con el mundo exterior.

a) Interfaz en una clase. - La interfaz en una clase representa un contrato de prestación de servicios entre ella y los demás componentes del sistema; de este modo, los clientes de un componente, solo necesitan conocer los servicios que éste ofrece y no cómo están implementados internamente, así se pude modificar la implementación de una clase sin afectar a las restantes relacionadas con ella.

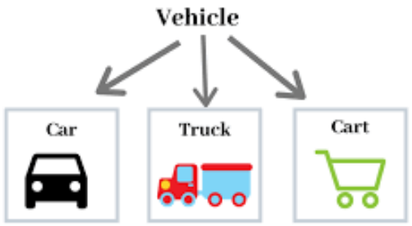
b) Existe una separación entre la interfaz y la implementación. - La primera establece qué se puede hacer con el objeto; de hecho, la clase actúa como una *caja negra;* es estable, la implementación se puede modificar.

Nota. - El encapsulamiento o encapsulación consiste en combinar datos y comportamiento en un paquete y ocultar los detalles de la implementación del usuario con el objeto.

2.1.3. Herencia

El concepto de clases divididas en subclases se usa en la vida diaria y conduce al de herencia; la clase de animal se divide en mamíferos, anfibios, insectos, aves, etc; la clase vehículo se divide en autos, camiones, autobuses, motocicletas, etc,

Ejemplo: La idea principal de estas divisiones reside en el hecho de que cada subclase (un camión, un autobús) comparte características con la clase de la cual se derivan (un vehículo): todos tienen moto, ruedas y frenos; **pero** además de estas que comparten cada subclase tiene propias características: un auto puede tener maletero, cinco asientos; los camiones, cabina y caja para transportar carga y otras características.



La clase principal de la que se derivan las restantes se denomina **base, padre o superclase**; las subclases también se denominan **hijas o derivadas.**

Las clases reflejan que el mundo real contiene objetos con propiedades o atributos y comportamiento; la herencia manifiesta que dichos objetos tienden a organizarse en jerarquías; esta jerarquía desde el punto de vista del modelado, se denomina ***relación de generalización*** *o herencia* ***(***"**es un**").

El programador puede definir una clase Animal que encapsule todas las propiedades o atributos (altura, peso, número de patas, etc) y el comportamiento u operaciones (comer, dormir, andar) que pertenecen a cada uno. Un animal como un mono, jirafa, canguro o pingüino tienen sus propias características a su vez.

→ Un mono ***es-un*** Animal con características propias: subir árboles, saltar entre ellos; además comparte con otros animales como jirafa, canguro y pingüino las características de comer, beber, correr, dormir, y demás.

Nota. - La herencia permite la creación de nuevas clases a partir de otra ya existente; la clase que sirve de modelo se llama *base* y la que se creó ***hereda*** sus características propias, además se puede personalizar añadiendo rasgos adicionales.

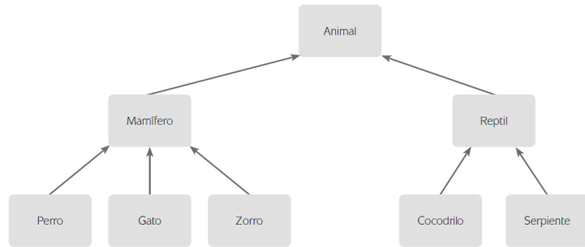
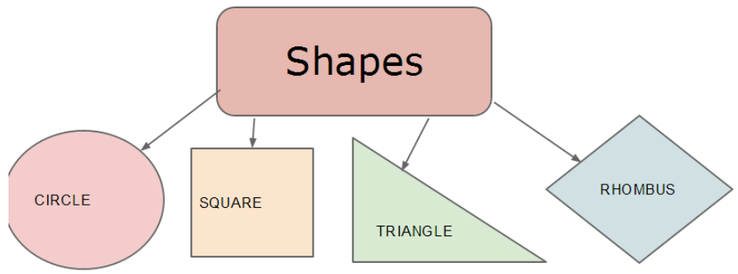


Figura. Herencia de la clase Animal
Joyanes, Programación en Java, p. 175

2.1.4. Polimorfismo

Es la propiedad que le permite a una operación o función tener el mismo nombre en clases diferentes y actuar de modo distinto en cada una de ellas: se puede abrir una puerta, una ventana, un libro, un periódico; en cada caso se realizar una acción diferente; **cada clase "conoce" cómo realizar esa operación.**

Ejemplo: Si se quiere hacer un diseño gráfico para representar figuras geométricas como triángulo, rectángulo y círculo y desea calcular sus superficies y perímetros; cada figura tiene un método u operación distinta para realizar dichos cálculos, el polimorfismo permite definir la única función o método calcularSuperficie, cuya implementación es diferente en la clase triangulo, rectángulo o círculo, y cuando se selecciona un objeto específico la función que se ejecuta es la correspondiente al objeto de la clase seleccionada.



Nota. - El uso de funciones u operadores de forma diferente, dependiendo de los objetos sobre los que actúan, se denomina polimorfismo. (un elemento con diferentes formas)

2.1.5. Reutilización (reusabilidad)

Este concepto significa que una vez que se ha creado, escrito y depurado una clase, se puede poner a disposición de otros programadores; de manera similar, al uso de las bibliotecas en un lenguaje de programación. El concepto de herencia proporcionará una ampliación o extensión importante a la idea de *reusabilidad;* una clase existente se puede ampliar añadiéndole nuevas características, atributos y operaciones.

2.2. Clases

Una clase es una plantilla, modelo o plano a partir de la cual se crean objetos; los que se derivan de la misma clase tienen características o propiedades similares; cuando se construye un objeto de una clase, se dice que se ha creado una *instancia, ejemplar o copia* de esta.

Aunque en un lenguaje orientado a objetos contenga centenares o miles de clases que sirven para diversos propósitos, cálculos matemáticos, cálculo de fechas y calendarios, programación de redes y aplicaciones diversas de toda índole. Sin embargo, cuando desarrolle un programa, además de las clases de las bibliotecas, necesitarás crear clases propias que describan los objetos del dominio del problema de sus aplicaciones.

Los programadores se deben centrar en la creación de clases; cada una contiene campos de instancia y datos de un objeto; los procedimientos o funciones que operan sobre los datos que se llaman *métodos.* Un objeto que es una instancia de una clase tendrá valores específicos para sus campos de instancia *atributos.*

2.3. Objetos

El mundo real está plagado de objetos: personas, animales, plantas, automóviles, edificios, computadoras, teléfonos, casas, semáforos, lápices, etc. Los objetos pueden interactuar entre sí y todos tienen tres características o propiedades clave:

$$objeto = estado + comportamiento + identidad$$

2.3.1. Estado

Los atributos son datos que diferencian un objeto de otro; el conjunto de ellos constituye su estado; cada objeto almacena información acercad de su estado actual y en un momento dado éste corresponde a una selección determinada de valores posibles de los diversos **atributos**. Estos últimos son propiedades o características de una clase y describen un rango de valores; un objeto y su clase asociada podrán contener varios atributos o ninguno.

Este estado puede cambiar con el tiempo como consecuencia de llamadas a métodos; si dicho estado cambiara sin una llamada de un método se rompería el principio de encapsulamiento de datos.

2.3.2. Comportamiento

El conjunto de capacidades y aptitudes que describen sus operaciones, funciones y reacciones; además responde a lo que se puede hacer con dicho objeto o a los métodos que se le pueden aplicar. Todos los objetos de una misma clase soportan el mismo comportamiento, el cual se define por los métodos que se pueden llamar y utilizar para cambiar sus atributos, recibir información de otros objetos y enviar mensajes solicitando la ejecución de tareas. Los métodos se llaman funciones o procedimientos y son los grupos de sentencias o instrucciones relacionadas de una clase que realizan o ejecutan una tarea específica sobre sus propios objetos.

2.3.3. Identidad

Es la propiedad que diferencia un objeto de otro similar; su estado no describe totalmente al objeto ya que cada uno tiene distinta identidad. Durante la fase de implementación, la identidad se crea, normalmente usando un identificador que viene del dominio del problema; por ejemplo, los automóviles tienen un número de la placa; los teléfonos tienen un número, aunque una misma persona pueda tener varios: el personal, del trabajo, de casa, etc.

2.3.4. Cómo identificar las clases y objetos

La regla más simple para identificar **clases** es buscar **nombres** en el análisis del problema; los **métodos** por el contrario corresponden a **verbos**.

Ejemplo: En el caso de un sistema de pedidos a un almacén, los nombres pueden ser: artículo, pedido, dirección de envío, sistemas de pago, cuanta banco; sus verbos serían los métodos en consecuencia hay que buscarlos en la definición del problema: *añadir* artículos al pedido, *solicitar* un pedido, *cancelarlo, cambiar* de dirección, *elegir* un sistema de pago, etc.

Además de introducir clases, objetos y métodos, se requiere añadir **atributos** como nombre, cliente, dirección, edad número de pasaporte o precio del artículo. Estos atributos se especifican como una parte de la clase del objeto y se representan con *variables de instancia* o, simplemente, *variables de la clase.*

Naturalmente, la regla "*encontrar verbos y nombres*" es práctica simple a medida que el programador adquiera experiencia aprenderá a seleccionar los verbos y nombres que serán clases para construir las nuevas.

2.3.5. Relaciones entre clases

Una relación es una conexión semántica entre clases que permite que una conozca los atributos, operaciones y relaciones de otras, debido a que las clases no actúan aisladas, sino relacionadas. La fortaleza de una relación se basa en el modo de dependencia de las clases implicadas en las relaciones entre ellas; por ejemplo, si dos clases son bastante dependientes una de otra, están **acopladas fuertemente**; en caso contrario **estarán acopladas débilmente**.

Las relaciones entre clases son similares a las relaciones entre objetos físicos del mundo real o entre objetos imaginarios en un mundo virtual; en UML, las formas en las que se conectan entre sí las clases, lógica o físicamente, se modelan como relaciones. En el modelado orientado a objetos existen tres clases de relaciones importantes: dependencias, generalizaciones-especializaciones, asociaciones-agregaciones.

a) Las **dependencias** son relaciones de uso (***usa un***). Declara que un elemento utiliza la información y los servicios de otro, pero no a la inversa necesariamente. Una clase depende de otra si sus métodos usan o manipulan objetos de ésta. La clase pedido usa la clase cuenta porque sus objetos necesitan acceder a los de cuenta para comprobar el estado de saldo, pero la clase artículo, aunque se incluyen en el pedido, no depende de cuenta, ya que los diferentes elementos del pedido, no necesitan conocer la cuenta del cliente. Las dependencias implican que los objetos de una clase pueden trabajar juntos, por consiguiente, se considera que es la relación directa más débil que existe entre dos clases; por esta razón debe minimizar el número de clases que depende de otras (*minimizar el acoplamiento entre clases*).
b) Las **asociaciones** son relaciones estructurales entre objetos; una asociación *todo/parte,* en la cual una clase representa algo grande, el todo, que consta de elementos más pequeños o las partes se denomina *agregación.* (***tiene-un***). Una relación de *todo y parte* o *compuesto y componentes;* un objeto de la clase A contiene objetos de la clase B; un objeto de pedido cuenta con elementos de artículo. Una agregación es un caso especial de asociación, la cual es más fuerte que la dependencia y normalmente indica que una clase recuerda o retiene una relación con otra durante un periodo determinado; es decir, las clases se conectan conceptualmente en una asociación: un jugador es parte de un equipo, una persona posee un carro, etc.
c) Las **generalizaciones** conectan clases generales con otras más especializadas en lo que se conoce como relaciones *subclase/superclase* o *hijo/padre*. (***es-un***). La herencia es una relación expresada entre una clase más especial y una más general denominada base; la clase creada a partir de ella se llama *derivada.* En general, si la clase A se extiende o deriva de la clase B, la primera hereda métodos de la segunda, pero tiene más características o funcionalidades.

2.3.6. UML: Modelado de aplicaciones

El lenguaje unificado de modelado, es el lenguaje estándar de modelado *de facto* para el desarrollo de sistemas y *software*, cuya popularidad creció en el diseño de otros dominios. Es muy aplicable en la representación y creación de la información que se utiliza en las fases de análisis y diseño; en diseño de sistemas, se modela por una importante razón: ***gestionar la complejidad.***

a) Modelo: Es la abstracción de cosas reales, cuando se modela un sistema, se realiza una abstracción ignorando los detalles que sean irrelevantes, con un lenguaje de modelado formal, aunque abstracto es tan preciso como un lenguaje de programación.

¿Qué es UML? Es un lenguaje que tiene sintaxis como semántica y se compone de un pseudocódigo, código real, dibujos, programas, descripciones, etc. Los elementos que constituyen un lenguaje de modelado se denominan **notación.** Un lenguaje de modelado puede ser cualquier entidad con una notación como medio de expresión del modelo y una descripción de lo que significa esa notación (metamodelo), las ventajas de UML son:

- Diseño de *software*
- *Software* de comunicaciones
- Proceso de negocios
- Captura de detalles acerca de un sistema, proceso u organización en análisis de requisitos
- Documentación de un sistema, proceso o sistema existente

2.3.7. Diseño y representación gráfica de clases y objetos en UML

El proceso de abstracción comienza con la identificación de las características comunes de un conjunto de elementos y prosigue con la descripción concisa de éstas en lo que convencionalmente se llama *clase.* Una clase describe el dominio de definición de un conjunto de objetos que le pertenecen; sus características generales están contenidas en la clase y las especializadas se encuentran en los objetos. Así una clase define la estructura y el comportamiento que compartirá un conjunto de objetos, cada objeto de una clase dada contiene estructura o estado y el comportamiento está definido por ambos; como se mencionó se conocen como instancias. **Una clase es una construcción lógica, mientras que un objeto tiene realidad física.**

Cuando se crea una clase, se especifica el código y los datos que la constituyen; de modo general, estos elementos se llaman *miembros*; los datos definidos se denominan variables miembro o de instancia; el código que opera sobre los datos se conoce como método miembro o simplemente, método, en la mayoría de las clases, las variables de instancia son manipuladas o accedidas por los métodos definidos por esa clase; por consiguiente, los métodos determinan cómo se pueden utilizar los datos de la clase.

Las variables definidas *dentro* de la clase se llaman de instancia debido a que cada instancia de la clase (cada objeto), tiene su propia copia de estas variables; entonces los datos de un objeto son independientes y únicos de los de otro objeto.

El propósito de una clase es encapsular complejidad, existen mecanismos para ocultar la complejidad de la implementación dentro de la clase; cada método o variable de una clase se puede señalar como público o privado; la interfaz pública de una clase representa todo o que los usuarios externos necesitan o pueden conocer; los métodos y los datos privados sólo pueden ser accedidos por el código que es miembro de la clase; así cualquier otro código no puede entrar a un método o variable privado. Como sólo se puede acceder a los miembros privados de una clase por otras partes del programa a través de métodos públicos de la misma, se puede asegurar que no sucederá ninguna acción no deseada; naturalmente, esto significa que la interfaz pública debe diseñarse cuidadosamente para no exponer a la clase de forma innecesaria.

Nota. –

- Una clase es una entidad que encapsula información y comportamiento.
- Un objeto es una instancia o ejemplar de una clase.

2.3.8. Representación gráfica de una clase

Una clase representa un concepto tangible o concreto, como un avión; o abstracto, como un documento; o intangible como las inversiones de alto riesgo; en UML, una clase es una caja rectangular dividida en compartimentos, secciones o bandas. Un compartimento es el área rectangular donde se escribe información: el

primero contiene el *nombre de la clase;* el segundo tiene los *atributos* y el tercero se usa para las *operaciones.* Se puede ocultar o quitar cualquier compartimento de la clase para aumentar la legibilidad del diagrama; cuando no existe alguno, no significa que esté vacío puesto que se pueden añadir para mostrar información adicional como excepciones o eventos, aunque no es muy común incluir estas propiedades. UML propone que una clase:

- Comience con letra mayúscula.
- Esté centrado en el compartimento o banda superior.
- Se escriba en tipo de letra o fuente negrita.
- Aparezca en cursiva cuando la clase sea abstracta.

Un objeto es una instancia de una clase; se pueden tener varias instancias de una clase llamada, por ejemplo, auto; como un Audi rojo, de dos puertas; un Renault azul, de cuatro puertas y un todo terreno, verde, de cinco puertas; cada instancia es un objeto al que se puede dar nombre o dejarlo anónimo y se representa en los diagramas de objetos.

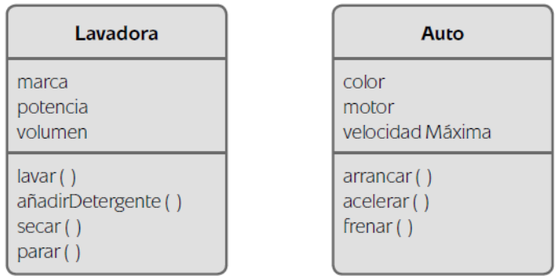
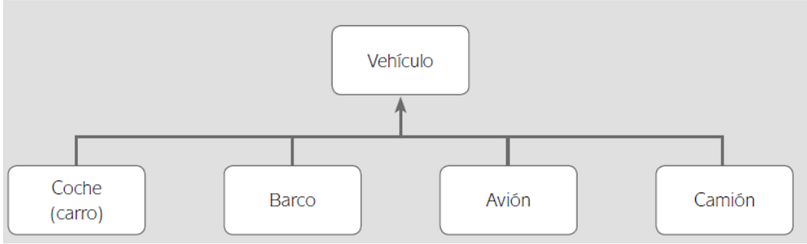


Figura. Diagrama de clases lavadora y auto
Programación en Java, Joyanes, p.190

La herencia es la manifestación más clara de la relación de generalización/especialización, una de las propiedades más importantes de la orientación a objetos, gráficamente podemos representar en UML la herencia como una línea que conecta padres con hijas; en la parte de la línea que las conecta se pone un triángulo abierto o punta de flecha apuntando a la clase padre; este tipo de conexión se representa como *es-un.* En caso de que una clase no tenga padre, se denomina *base o raíz;* si no tiene ninguna clase hija, se denomina *terminar o hija*; si tiene un padre exactamente es de herencia simple; si tiene más de un padre, es de herencia múltiple. Una clase hija también puede ser padre de otra clase hija; por ejemplo, mamífero es hija de animal y padre de caballo.



Jerarquía Tipo Árbol, Herencia de clase vehículo
Programación en Java, Joyanes, p.202

Como base y derivada tienen código y datos comunes, **si la derivada se creara de modo independiente**, se duplicaría mucho de lo que ya se ha escrito para la base.

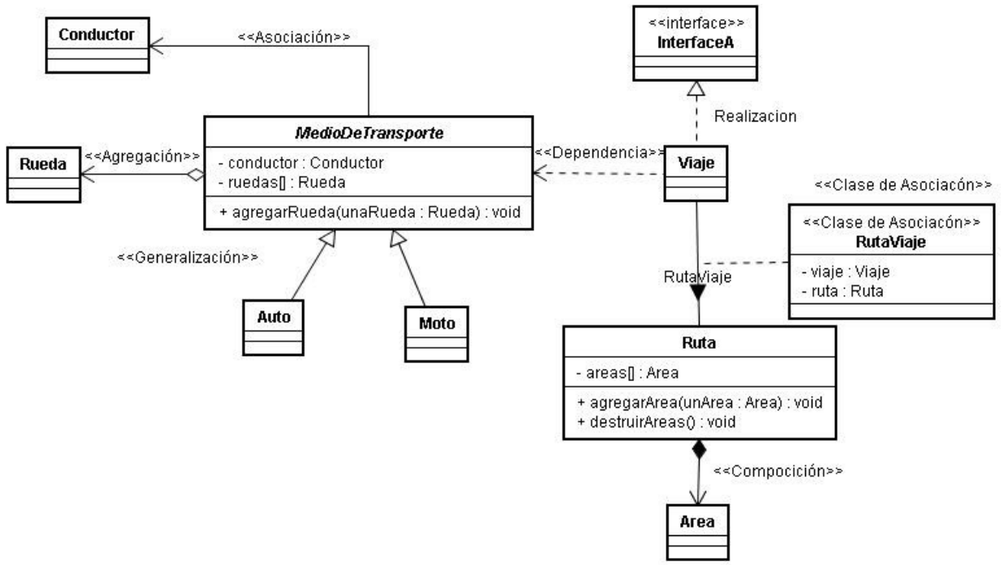


Figura. Diagrama ejemplo de implementación de abstracción "Empresa de transporte interdepartamental y departamental"

PARTE II

1. **Declaración de una clase**

Se debe considerar el ***nombre de la clase***, ésta debe definirse, lo que implica darle un nombre a la clase y a los ***elementos*** que almacenan sus datos, así como describir los ***métodos*** que realizarán las acciones consideradas en los objetos.

1.1. **Constructor**

Un constructor es un método que se ejecuta automáticamente cuando se crea un objeto de una clase; sirve para inicializar los miembros de la misma. El proceso de instanciar un objeto de una clase, se puede realizar con el método llamado __init__

1.2. **Self**

El primer argumento cuando definimos cualquier método siempre estará el *self*, este argumento especifica la instancia (el objeto con el que se está trabajando) en el cual se llama el método. Self le indica al intérprete de Python la información sobre una instancia en concreto, para definir un método se usa self para modificar los atributos de la instancia actual.

2. **Ejemplo 1 Clase Punto**

Se instanciará una clase Punto que contiene coordenadas x-y de un punto en el plano cartesiano.

2.1. **Dar nombre a la clase**. - En Python la palabra reservada para crear una clase es **class** seguido del nombre de la clase y sus dos puntos (al igual como se trabajaba con funciones, condicionales o bucles en Python), siendo muy recomendable que el nombre empiece con la letra mayúscula.

```
class Punto:
```

2.2. **Constructor y Atributos**. – Definiremos 2 atributos x e y que guardarán el valor de coordenadas que tendrá cada instancia de la clase Punto. Esto se desarrollará en un constructor __init__ donde por parámetros le pasaremos las variables x e y que definiremos al instanciar un objeto. La palabra reservada **self** como se dijo indica que modificaremos las variables de la instancia indicándole que esa información corresponde a ese objeto en concreto.

```
class Punto:
    def __init__(self, x, y):
        self.x=x
        self.y=y
```

Los atributos instanciados tendrán el valor que se asignará en los parámetros del constructor de la clase; por lo tanto, estos atributos se denominan atributos de instancia. En la siguiente imagen se mostrará una instancia de la clase Punto pasando valores a los argumentos x e y.

```
punto1=Punto(2, 3)
```

El modo de instanciar un objeto de la clase es primero dándole el nombre al objeto en este caso "punto1" seguido del operador de asignación y el nombre de la clase, dentro del paréntesis se inicializarán los atributos con los valores de x e y mostrados en el constructor __init__.

2.3.**Método mostrar coordenada X e Y**. – Para mostrar los valores que le dimos a cada coordenada dentro de la clase desarrollaremos un método que al llamarlo muestre ambas coordenadas, quedando la clase como:

```
class Punto:
    def __init__(self, x, y):
        self.x=x
        self.y=y

    def mostrarCoordenadas(self):
        print(f'(x={self.x}, y={self.y})')
```

La forma de poder llamar al método "mostrar coordenadas" será escribiendo el nombre del objeto seguido de un punto y el nombre del método desarrollado, como se muestra a continuación:

```
punto1=Punto(2, 3)

punto1.mostrarCoordenadas()
```

Como resultado nos mostrará las coordenadas en x e y del objeto punto1 que acabamos de instanciar.

```
(x=2, y=3)
```

Realizando los mismos pasos de la instanciación podemos crear varios objetos de la clase punto con otros valores.

```
punto1=Punto(2, 3)
punto2=Punto(4, 5)
punto3=Punto(9, -1)
punto1.mostrarCoordenadas()
punto2.mostrarCoordenadas()
punto3.mostrarCoordenadas()
```

```
In [16]: runfi
wdir='C:/Users
(x=2, y=3)
(x=4, y=5)
(x=9, y=-1)
```

2.4.**Modificación de coordenadas una vez instanciado el objeto.** - Del mismo modo en el que pudimos instanciar cada objeto de la clase Punto podemos modificar sus atributos.

```
print("punto3")
punto3.x=100
punto3.mostrarCoordenadas()
```

Como resultado se habría modificado el punto3 la coordenada x:

```
punto3
(x=100, y=-1)
```

3. **Visibilidad de los miembros de la clase**

Un principio fundamental en programación orientada a objetos es la ocultación de la información; no poder acceder por métodos externos a la clase a determinados datos internos; el mecanismo para ocultar datos es ponerlos en una clase y hacerlos privados; a éstos solo se podrá acceder desde el interior de la clase. Por el contrario, los datos o métodos públicos son accesibles desde el exterior de la clase.

Declarando en el constructor los atributos **privados** no se podrá acceder y modificar los valores como se hizo en el punto 2.4.

En Python la forma de hacer que un atributo sea privado es con un doble guion bajo en cada atributo (y con un solo guion bajo se vuelve un atributo protegido, en caso de atributo público no se pone nada).

```
class Punto:
    def __init__(self, x, y):
        self.__x=x
        self.__y=y

    def mostrarCoordenadas(self):
        print(f'(x={self.__x}, y={self.__y})')
```

Volviendo los puntos x e y protegidos solo se podrá acceder a cada uno de ellos dentro de la clase como el método mostrarCoordenadas() puede acceder a los atributos x e y. la situación de la ejecución será la misma en la instanciación, pero no se podrá modificar el valor x o y como se hizo en el punto 2.4.

```
print("punto3")
punto3.x=100
punto3.mostrarCoordenadas()
```

```
(x=9, y=-1)
punto3
(x=9, y=-1)
```

Como se muestra en la imagen superior x no cambió de valor como lo hacía anteriormente, incluso si queremos acceder de la misma manera que lo hace el método de la clase mostrarCoordenadas con dos guiones bajos, no se modificará el valor x.

```
print("punto3")
punto3.__x=100
punto3.mostrarCoordenadas()
```

```
(x=9, y=-1)
punto3
(x=9, y=-1)
```

3.1.**¿Cuándo usar privado, público y protegido?** El uso corresponderá del análisis previo realizado sobre el problema, pero un resumen breve sería:

- Publico. – Define miembros públicos, aquellos a los que se puede acceder por cualquier método desde fuera de la clase (como lo hicimos en el ejemplo 4)
- Protegido. – Se puede acceder a estos por aquellos miembros de la misma clase o de **clases derivadas**, así como por **métodos** de otras clases que se encentren en el mismo paquete (Archivo donde se almacenan todas las clases .py separadas)
- Privado. – Se puede acceder a estos por métodos de la **misma clase** únicamente.

De la misma manera tanto clases como métodos pueden instanciarse de cualquier tipo de especificador dependiendo del análisis realizado.

La labor más importante de estos especificadores de acceso es implementar la ocultación de la información; este principio indica que toda la interacción con un objeto se debe restringir al uso de una interfaz bien definida que permita que los detalles de implementación de los objetos sean ignorados; por consiguiente, los datos y métodos públicos forman la interfaz externa del objeto, mientras que los elementos privados son los aspectos internos que no necesitan ser accesibles para su uso.

3.2.**¿Cómo obtengo u modifico el valor de un atributo?**

El significado de encapsulación, es hacer que los datos sean seguros y ocultos del usuario, esto se consigue con los especificadores de acceso o proveer métodos públicos **get** (obtener) y **set** (modificar) para acceder y actualizar el valor de atributos no públicos. Para poder modificar el atributo privado x de la Clase debemos desarrollar estos métodos que nos permitan acceder a datos y poder modificarlos en *jerga* de POO **getters y setters** como se muestra a continuación.

```
## Getters
def obtenerX(self):
    return self.__x
def obtenerY(self):
    return self.__y
## Setters
def modificarX(self, x):
    self.__x=x
def modificarY(self, y):
    self.__y=y
```

De esta forma no se rompe la encapsulación y solo deberemos usarlos en caso necesario. Ahora usando el método publico modificarX() en lugar de acceder a la variable privada podremos modificar el valor del atributo.

4. **Métodos de una clase**

Son miembros de clases, no hay métodos o funciones fuera de ellas; su implementación se incluye dentro del cuerpo de la clase.

4.1.**Ejemplo 2 Fecha de cumpleaños**

Definir una clase Fecha que contenga los atributos mes y día, los métodos igual() y visualizar() para luego verificar si una fecha es cumpleaños de alguien. La estructura de nuestra clase quedará de la siguiente manera:


```
class Fecha:
    def __init__(self, dia, mes):
        self.__dia=dia
        self.__mes=mes
    ## Getters
    def obtenerDia(self):
        return self.__dia
    def obtenerMes(self):
        return self.__mes
    ## Metodos
    def igual(self, fecha):
        if fecha.obtenerDia()==self.__dia
        and fecha.obtenerMes()==self.__mes:
            return True
        else:
            return False
    def visualizar(self):
        print(f'mes={self.__mes}, dia={self.__dia}')
```

En comentario dejamos los *getters* que necesitaremos para poder acceder a los atributos de otro objeto Fecha instanciado y usado en el método **igual.**

Nos piden que verifiquemos que dado un día es nuestro cumpleaños o no, fuera de la clase, instanciaremos dos Fechas la del día de hoy y el de nuestro cumpleaños usaremos el método visualizar que nos mostrará el mes y día de nuestros objetos instanciados, para luego desarrollar una estructura condicional haciendo uso del método igual que comparará dos objetos de tipo Fecha.

```
hoy=Fecha(30, "Mayo")
cumple=Fecha(30, "Mayo")
print("La fecha de hoy es: ")
hoy.visualizar()
print("Tu fecha de nacimiento es: ")
cumple.visualizar()
if hoy.igual(cumple):
    print("Feliz Cumpleaños 😊")
else:
    print("Todavía falta para tu cumpleaños 😔")
```

Cómo resultado de los valores instanciados en el objeto obtendremos:

```
La fecha de hoy es:
mes=Mayo, dia=30
Tu fecha de nacimiento es:
mes=Mayo, dia=30
Feliz Cumpleaños 😊
```

En cambio, si el día de hoy no fuese el de nuestro cumpleaños, veremos:

```
La fecha de hoy es:
mes=Julio, dia=1
Tu fecha de nacimiento es:
mes=Mayo, dia=30
Todavía falta para tu cumpleaños😔
```

5. **Paquetes y Clases predefinidos en Python**

Los paquetes es la forma que tienen algunos lenguajes de organizar los archivos con las clases necesarias para construir las aplicaciones, en Python anteriormente hemos hecho uso de algún paquete como math o datetime, para obtener el logaritmo de un número o le hora, día, mes, etc., de una fecha. En cada caso hemos importado una clase y obtenido los métodos y atributos que requeríamos, como se muestra en el ejemplo del *módulo* datetime importaremos la clase *datetime* para usar el método now() que obtendrá la fecha y hora actual.

```
from datetime import datetime

x=datetime.now()
print(x)
```

```
2022-07-04 00:24:43.186489
```

El uso de métodos es el mismo que realizamos cuando llamábamos a uno de nuestras clases implementadas, muchos lenguajes de programación tienen esta característica y puede o no ser visibles las clases donde se obtienen estos métodos y cómo implementan su estructura.


```
class datetime(date):
    """datetime(year, month, day[, hour[, minute[, second[, microsecond[,tzinfo]]]]])

    The year, month and day arguments are required. tzinfo may be None, or an
    instance of a tzinfo subclass. The remaining arguments may be ints.
    """
    __slots__ = date.__slots__ + time.__slots__

    def __new__(cls, year, month=None, day=None, hour=0, minute=0, second=0,
                microsecond=0, tzinfo=None, *, fold=0):
        if (isinstance(year, (bytes, str)) and len(year) == 10 and
            1 <= ord(year[2:3])&0x7F <= 12):
            # Pickle support
            if isinstance(year, str):
                try:
                    year = bytes(year, 'latin1')
                except UnicodeEncodeError:
                    # More informative error message.
                    raise ValueError(
                        "Failed to encode latin1 string when unpickling "
                        "a datetime object. "
                        "pickle.load(data, encoding='latin1') is assumed.")
            self = object.__new__(cls)
```

Internamente es una clase llamada datetime, con sus métodos y atributos establecidos, el método que usamos now() estará en la clase.

```
def now(cls, tz=None):
    "Construct a datetime from time.time() and optional time zone info."
    t = _time.time()
    return cls.fromtimestamp(t, tz)
```

Si apreciamos el mismo método now está consumiendo otro método de la clase pasando argumentos y retornando el valor, el atributo *time* está protegido y es uso del módulo y las clases que contiene como se muestra en el código. Adicional es muy recomendable añadir un comentario describiendo que realiza el método ya que nosotros no seremos los únicos que usarán el método.

6. **Tipos de datos Abstractos**

Ya conocemos los datos primitivos como los enteros, booleanos, doubles, long y char, pero un tipo de dato abstracto describe un conjunto de objetos con la misma representación y comportamiento; por consiguiente, se puede usar implementaciones alternativas para el mismo tipo abstracto de dato sin cambar su interfaz, esto se hace mediante las clases. Esto lo podemos realizar en Python con el método type(), que se muestra en el siguiente ejemplo:

```
a=34
print(type(a))
hoy=Fecha(1, "Julio")
print(type(hoy))
```

```
<class 'int'>
<class '__main__.Fecha'>
```

Cómo se muestra en la imagen al imprimir el tipo de dato que es a nos muestra que es de tipo int o entero, pero al tratar de imprimir el tipo de dato de Fecha nos muestra que es de tipo Fecha, siendo este un tipo de dato abstracto.

7. **Herencia**

La herencia o relación *es-un*, es la relación existente entre dos clases: una derivada que se crea a partir de otra ya existente, denominada *base*; la nueva hereda de la ya existente.

Ejemplo: Se declararán las clases Persona que tiene primer nombre y primer apellido como atributos y un método que imprime su nombre.

```
class Persona:
    def __init__(self, primerNombre, primerApellido):
        self.primerNombre=primerNombre
        self.primerApellido=primerApellido
    def mostrarNombre(self):
        print(f'{self.primerNombre} {self.primerApellido}')

persona1=Persona("Jaime", "Perez")
persona1.mostrarNombre()
```

Luego se declarará una clase Estudiante que heredará los atributos de persona pero que además modificará el constructor del padre agregando sus propias particularidades, en este caso la fecha de ingreso a su institución.

super() Se puede llamar a los métodos heredados de la clase base desde cualquier método de la clase derivada, simplemente se escribe su nombre y la lista de argumentos; puede ocurrir que haya métodos de la clase base que no interesa heredar en la derivada, debido a que se busca una funcionalidad adicional; para ellos se sobreescribe el método en la clase derivada, los métodos en la clase derivada de mismo nombre se anulan o reemplazan al método de la base, el método ocultado de la base puede llamarse desde cuaquier método de la derivada mediante la referencia super, En python y java la función super nos permite acceder a cualquier miembro de la clase base siempre que no sea privado.

La forma en la que la clase Esudiante hereda de la clase Persona es que en los parentesis se escribe la clase de la que heredará, note que en el constructor __init__ de la clase estudiante usamos la función super() que invoca al padre su constructor añadiendo los datos respectivos del primer nombre como el apellido, como resultado tendremos:

```
class Estudiante(Persona):
    def __init__(self, primerNombre, primerApellido, anio):
        super().__init__(primerNombre, primerApellido)
        self.anioIngreso=anio

    def bienvenido(self):
        print(f"Bienvenido {self.primerNombre}, {self.primerApellido} del año

estudiante1=Estudiante("Federico", "Buendia", 2020)
estudiante1.bienvenido()
estudiante1.mostrarNombre()
```

Observe que el método mostrarNombre() no esta en la clase Estudiante, lo que sucede es que la esta heredando de su clase padre para la muestra del nombre y apellido del Estudiante, la ejecución de este código sería:


```
Bienvenido Federico, Buendia del año 2020
Federico Buendia
```

Nota.- La herencia siempre es simple y pública; la clase derivada hereda todos los miembros de la clase basea excepto los miembros privados; en el paradigma de la orientación a objetos, para que un lenguaje sea considerado de ese tipo debe soportar la propiedad de la herencia. Un objeto de una clase derivada se crea siguiendo este orden: primero la parte del objeto correspondiente a la base y a continuación se crea la parte propia de la derivada; para llamar al constructor de la primera desde el constructor de la última se emplea la palabra reservada super().

¡Gracias por leer este resumen, evidentemente faltan muchas cosas por descubrir en la programación orientada a objetos, espero que esta sea una base para iniciar😊! Psdt. Hay miles de videos que puedes ver y poner en práctica los conceptos aprendidos aquí; ve y míralos seguro serán más aclarativos y podrás comprender mejor este manual.